

6.2.4 Stability of a numerical algorithm ¶

6.2.4 Numerical stability

fit width



Correctness in the presence of error (e.g., when floating point computations are performed) takes on a different meaning. For many problems for which computers are used, there is one correct answer and we expect that answer to be computed by our program. The problem is that most real numbers cannot be stored exactly in a computer memory. They are stored as approximations, floating point numbers, instead. Hence storing them and/or computing with them inherently incurs error. The question thus becomes "When is a program correct in the presence of such errors?"

Let us assume that we wish to evaluate the mapping $f : \mathcal{D} \rightarrow \mathcal{R}$ where $\mathcal{D} \subset \mathbb{R}^n$ is the domain and $\mathcal{R} \subset \mathbb{R}^m$ is the range (codomain). Now, we will let $\tilde{f} : \mathcal{D} \rightarrow \mathcal{R}$ denote a computer implementation of this function. Generally, for $x \in \mathcal{D}$ it is the case that $f(x) \neq \tilde{f}(x)$. Thus, the computed value is not "correct". From earlier discussions about how the condition number of a matrix can amplify relative error, we know that it may not be the case that $\tilde{f}(x)$ is "close to" $f(x)$: even if $\tilde{f}$ is an exact implementation of f , the mere act of storing x may introduce a small error δx and $f(x + \delta x)$ may be far from $f(x)$ if f is ill-conditioned.

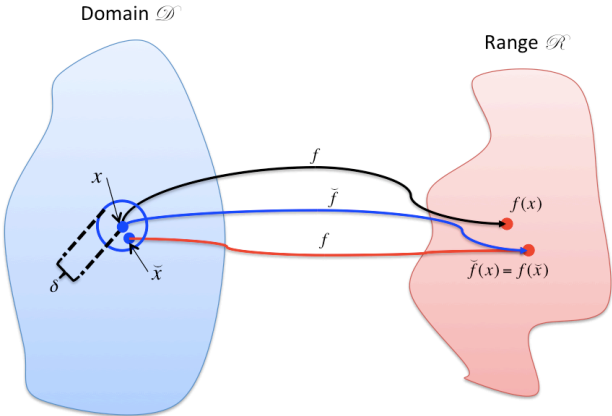


Figure 6.2.4.1. In this illustration, $f : \mathcal{D} \rightarrow \mathcal{R}$ is a function to be evaluated. The function $\tilde{f}$ represents the implementation of the function that uses floating point arithmetic, thus incurring errors. The fact that for a nearby value, $\tilde{x}$, the computed value equals the exact function applied to the slightly perturbed input x , that is,

$$f(\tilde{x}) = \tilde{f}(x),$$

means that the error in the computation can be attributed to a small change in the input. If this is true, then $\tilde{f}$ is said to be a (numerically) stable implementation of f for input x .

The following defines a property that captures correctness in the presence of the kinds of errors that are introduced by computer arithmetic:

Definition 6.2.4.2. Backward stable implementation. Given the mapping $f : \mathcal{D} \rightarrow \mathcal{R}$, where $\mathcal{D} \subset \mathbb{R}^n$ is the domain and $\mathcal{R} \subset \mathbb{R}^m$ is the range (codomain), let $\tilde{f} : \mathcal{D} \rightarrow \mathcal{R}$ be a computer implementation of this function. We will call $\tilde{f}$ a backward stable (also called "numerically stable") implementation of f on domain $\mathcal{D}$ if for all $x \in \mathcal{D}$ there exists a $\tilde{x}$ "close" to x such that $\tilde{f}(x) = f(\tilde{x})$.

In other words, $\tilde{f}$ is a stable implementation if the error that is introduced is similar to that introduced when f is evaluated with a slightly changed input. This is illustrated in [Figure 6.2.4.1](#) for a specific input x . If an implementation is not stable, it is numerically unstable.

The algorithm is said to be forward stable on domain $\mathcal{D}$ if for all $x \in \mathcal{D}$ it is that case that $\tilde{f}(x) \approx f(x)$. In other words, the computed result equals a slight perturbation of the exact result.

Example 6.2.4.3. Under the SCM from the last unit, floating point addition, $\kappa := \chi + \psi$, is a backward stable operation.

▼ [Solution](#)

$$\begin{aligned}
\tilde{\kappa} &= \text{< computed value for } \kappa \text{ >} \\
[\chi + \psi] &= \text{< SCM >} \\
(\chi + \psi)(1 + \epsilon_+) &= \text{< distribute >} \\
\chi(1 + \epsilon_+) + \psi(1 + \epsilon_+) &= \\
(\chi + \delta\chi) + (\psi + \delta\psi)
\end{aligned}$$

where

- $|\epsilon_+| \leq \epsilon_{\text{mach}}$,
- $\delta\chi = \chi\epsilon_+$,
- $\delta\psi = \psi\epsilon_+$.

Hence $\tilde{\kappa}$ equals the exact result when adding nearby inputs.

Homework 6.2.4.1.

- ALWAYS/SOMETIMES/NEVER: Under the SCM from the last unit, floating point subtraction, $\kappa := \chi - \psi$, is a backward stable operation.
- ALWAYS/SOMETIMES/NEVER: Under the SCM from the last unit, floating point multiplication, $\kappa := \chi \times \psi$, is a backward stable operation.
- ALWAYS/SOMETIMES/NEVER: Under the SCM from the last unit, floating point division, $\kappa := \chi / \psi$, is a backward stable operation.

▼ [Answer](#) ▼ [Solution](#)

- ALWAYS: Under the SCM from the last unit, floating point subtraction, $\kappa := \chi - \psi$, is a backward stable operation.
- ALWAYS: Under the SCM from the last unit, floating point multiplication, $\kappa := \chi \times \psi$, is a backward stable operation.
- ALWAYS: Under the SCM from the last unit, floating point division, $\kappa := \chi / \psi$, is a backward stable operation.

Now prove it!

- ALWAYS: Under the SCM from the last unit, floating point subtraction, $\kappa := \chi - \psi$, is a backward stable operation.

$$\begin{aligned}
\tilde{\kappa} &= \text{< computed value for } \kappa \text{ >} \\
[\chi - \psi] &= \text{< SCM >} \\
(\chi - \psi)(1 + \epsilon_-) &= \text{< distribute >} \\
\chi(1 + \epsilon_-) - \psi(1 + \epsilon_-) &= \\
(\chi + \delta\chi) - (\psi + \delta\psi)
\end{aligned}$$

where

- $|\epsilon_-| \leq \epsilon_{\text{mach}}$,
- $\delta\chi = \chi\epsilon_-$,
- $\delta\psi = \psi\epsilon_-$.

Hence $\tilde{\kappa}$ equals the exact result when subtracting nearby inputs.

- ALWAYS: Under the SCM from the last unit, floating point multiplication, $\kappa := \chi \times \psi$, is a backward stable operation.

$$\begin{aligned}
\tilde{\kappa} &= \text{< computed value for } \kappa \text{ >} \\
[\chi \times \psi] &= \text{< SCM >} \\
(\chi \times \psi)(1 + \epsilon_\times) &= \text{< associative property >} \\
\chi \times \psi(1 + \epsilon_\times) &= \\
\chi(\psi + \delta\psi)
\end{aligned}$$

where

- $|\epsilon_\times| \leq \epsilon_{\text{mach}}$,
- $\delta\psi = \psi\epsilon_\times$.

Hence $\tilde{\kappa}$ equals the exact result when multiplying nearby inputs.

- ALWAYS: Under the SCM from the last unit, floating point division, $\kappa := \chi / \psi$, is a backward stable operation.

$$\begin{aligned}
\tilde{\kappa} &= \text{computed value for } \kappa > \\
[\chi/\psi] &< \\
&= \text{SCM} > \\
(\chi/\psi)(1 + \epsilon_j) &= \text{commutative property} > \\
\chi(1 + \epsilon_j)/\psi &= \\
(\chi + \delta\chi)/\psi
\end{aligned}$$

where

- $|\epsilon_j| \leq \epsilon_{\text{mach}}$,
- $\delta\chi = \chi\epsilon_j$,

Hence $\tilde{\kappa}$ equals the exact result when dividing nearby inputs.

Ponder This 6.2.4.2. In the last homework, we showed that floating point division is backward stable by showing that $[\chi/\psi] = (\chi + \delta\chi)/\psi$ for suitably small $\delta\chi$.

How would one show that $[\chi/\psi] = \chi/(\psi + \delta\psi)$ for suitably small $\delta\psi$?